MLOps mašininio mokymosi operacijų sistemos

2 savaitė: atkartojamumas ir eksperimentų sekimas dideliu mastu

Povilas Kvedaras
@PovilasKvedaras

Vilniaus universitetas
Matematikos ir informatikos fakultetas
Informatikos institutas

2026 m. rudens semestras

Šiandienos temos

Paskaita (45 min.)

1. Atkartojamumo krizė — su skaičiais
2. Run'as ir penkios versijuojamos dimensijos
3. MLflow: eksperimentų žurnalas — gyvai
4. „At scale“: daugelio vartotojų serveris — gyvai
5. DVC: duomenų baitai ir pipeline — gyvai
6. W&B, Hydra ir įrankių žemėlapis
7. Lineage grandinė, reprodukcijos SLA, checklist

Po pertraukos — laboratorinis darbas: tą patį pasistatote patys.

Atkartojamumo krizė

1. dalis — 2 savaitė

Priminimas iš 1 savaitės

Praeitą kartą matėme:

- ML sistemoje **tikras ML kodas — maža dalis** (Sculley: paslėpta techninė skola);
- **nedeterminizmas**: seed'ai, GPU operacijų eiliškumas, shuffle;
- kodas $\neq$ modelis: modelį apibrėžia ir duomenys, ir konfigūracija.

Šios savaitės klausimas

Kaip pasiekti, kad **bet kurį** eksperimentą — savo ar kolegos — būtų galima **atkurti** po savaitės, mėnesio, audito metu?

Kas yra atkartojamumas?

Apibrėžimas

Atkartojamumas (reproducibility) — gebėjimas pakartoti eksperimentą ir gauti **statistiškai ekvivalentų** rezultatą, naudojant tuos pačius duomenis, kodą, konfigūraciją ir aplinką.

ACM skiria tris lygmenis:

Lygmuo	**Kas kartoja?**	**Su kuo?**
Pakartojimas (repeatability)	tie patys žmonės	ta pati aplinka
Atkartojimas (reproducibility)	**kiti žmonės**	tie patys artefaktai
Replikacija (replicability)	kiti žmonės	nauja implementacija

Šios savaitės įrankiai sprendžia pirmuosius du; trečiasis — mokslo metodo klausimas.

Krizė yra išmatuota

Šaltinis	**Rezultatas**
Nature (2016), 1 576 mokslininkų	>70 % nepavyko atkartoti kolegų; >pusei — **savo pačių**
Gundersen, Kjensmo (AAAI 2018), 400 AI straipsnių	kodą dalinosi tik **8 %**
Raff (NeurIPS 2019), 255 ML straipsniai	vien iš teksto atkartoti pavyko 63,5 %
Kapoor, Narayanan (*Patterns* 2023)	leakage klaidos **17-oje** mokslo sričių, 294 publikacijos
Henderson et al. (2018)	tas pats RL kodas, **kiti seed'ai** — kiti rezultatai

Kodėl eksperimentai neatkartojami?

1. **Kodas versijuotas, duomenys — ne.** Git seka `.py`, bet ne 50 GB CSV.
2. **Aplinka nesufiksuota.** Kitos `scikit-learn`, PyTorch, CUDA versijos.
3. **Hiperparametrai išsibarstę.** Notebook'e, Slack žinutėse, atmintyje.
4. **Atsitiktinumas nekontroliuojamas.** Nenurodyti seed'ai, GPU nedeterminizmas.
5. **„Paskutinio gero run'o" sindromas.** Išsaugotas tik geriausias bandymas.

Pagrindinė mintis

`git commit` yra **būtina, bet nepakankama** sąlyga. Versijuoti reikia: **kodą + duomenis + konfigūraciją + aplinką + artefaktus**.

Kodėl tai rūpi ne tik mokslui?

- **Kaina.** Llama 3.1 405B mokymas — 30,84 mln. GPU valandų. Neatkartojamas eksperimentas su tokiu biudžetu — tiesioginis nuostolis.
- **Reguliavimas.** ES AI akto 12 str. — automatinis įvykių registravimas; IV priedas — dokumentacija apie mokymo duomenų kilmę ir versijas.
- **Auditas.** „Kuris modelis šiandien produkcijoje ir kaip tiksliai jis buvo apmokytas?“ — po šios paskaitos atsakysite per 30 sekundžių.

Pramonės kartelė

Google *ML Test Score* infrastruktūros testų sąrašo **pirmasis punktas**: „mokymas yra atkartojamas“ (Infra 1). Tai ne prabanga, o žemiausia kartelė.

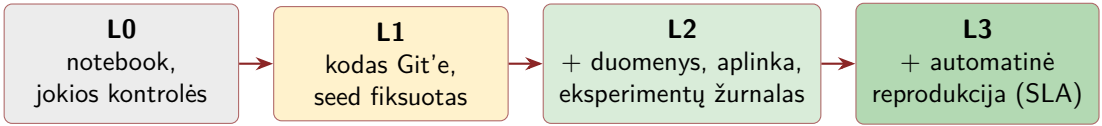
GPU determinizmas — kodėl „statistiškai ekvivalentus“?

Slankiojo kablelio sudėtis **nėra asociatyvi** — lygiagretus GPU vykdymas keičia sumavimo tvarką, todėl tas pats kodas su tuo pačiu seed'u gali duoti **bitais nesutampantį** rezultatą.

	CPU + sklearn	**GPU mokymas**
Tas pats seed	rezultatas sutampa **bitais**	gali nesutapti
Ką fiksuojame	tikslų atitikimą	**toleranciją**, pvz. $\Delta\text{accuracy} \leq 0{,}005$

PyTorch turi determinizmo režimą (`torch.use_deterministic_algorithms`), bet jis lėtina mokymą — todėl apibrėžime sakoma *statistiškai ekvivalentus*, o ne *identiškas*.

Atkartojamumo lygiai



Kurso tikslas: L2–L3. Šiandien pasistatysime visą kelią: žurnalas (MLflow), duomenų versijos (DVC), o pabaigoje — `reproduce.py` su išmatuojamu SLA.

Run'as ir penkios dimensijos

2. dalis — 2 savaitė

Eksperimentų sekimas = žurnalas (ledger)

Treniruodami modelį jūs **ieškote**: keičiate parametrus, duomenis, kodą. Paieška, kuri nepalieka pėdsakų, yra **prarasta informacija**.

Run'as

Kiekvienas paleidimas tampa žurnalo įrašu — **run'u** — su unikaliu ID, parametrais, metrikų istorija, žymomis ir artefaktais. Eksperimentas (experiment) — run'ų aplankas.

Visos vėlesnės kurso savaitės **skaito šitą žurnalą**:

- 6 sav. CI/CD vartai — ar metrikos leidžia release'ą?
- 7 sav. Registry — iš kurio run'o kilo diegiamas modelis?
- 11 sav. monitoringas — su kuo lyginti produkcijos elgseną?

Ką run'as fiksuoja — penkios dimensijos veiksme

Tipas	**Pavyzdžiai**	**Kodėl svarbu**
Parametrai	`n_estimators`, `random_seed`	nežinome, *su kuo* mokėta
Metrikos	accuracy, loss, F1	nežinome, *ar* modelis geras
Žymos (tags)	`git_commit`, `dvc_raw_md5`	run'as nesusietas su kodu ir duomenimis
Artefaktai	`model.pkl`, grafikai	negalime deploy'inti
Aplinka	Python, `scikit-learn==1.7`	fiksuojama **automatiškai** per `log_model`
Duomenys	rinkinio vardas + md5 santrauka	modelis mokosi kitko

Žymos — **jungties raktai**: jos susieja run'ą su Git commit'u ir DVC duomenų versija.

MLflow: eksperimentų žurnalas

3. dalis — 2 savaitė

MLflow žemėlapis

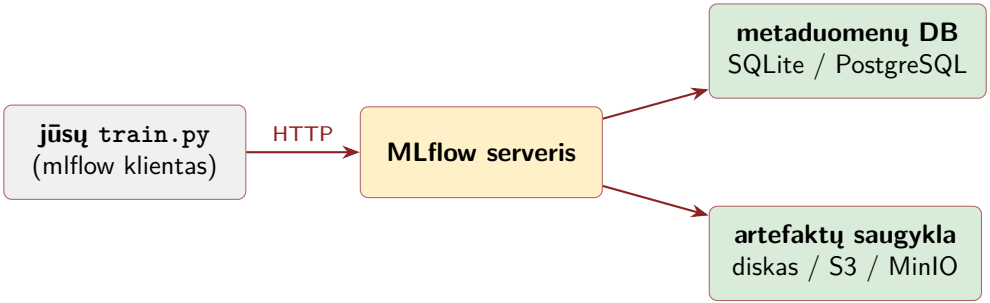
Sukurtas Databricks (2018), nuo 2020 — Linux Foundation; de facto atvirojo kodo standartas, kurio API kalba ir Azure ML, ir SageMaker.

Komponentas	**Ką daro**
Tracking	run'ų žurnalas: parametrai, metrikos, žymos, artefaktai
Models	universalus modelio formatas + aplinkos failai
Model Registry	vardinės versijos: `fraud-detector` v1, v2, v3
Server	UI + API — viena vieta visai komandai

Dėmesio

2025 m. MLflow perrašytas (3.0) — **beveik visi internete rasti pavyzdžiai yra iki-3.0 eros**. Kurso medžiaga atitinka MLflow 3.x.

Architektūra: klientas → serveris → dvi saugyklos



Parametrai ir metrikos gula į DB (jas reikia filtruoti), modeliai — į artefaktų saugyklą (jie dideli).

„At scale“ tezė

Skirtumas tarp „ant mano nešiojamo“ ir „visai komandai“ yra **vienas aplinkos kintamasis**: `MLFLOW_TRACKING_URI`. Kodas nesikeičia.

Run'as su visomis jungtimis (MLflow 3)

```
with mlflow.start_run(run_name="rf-n100-d5") as run:
    mlflow.log_params({"n_estimators": 100, "max_depth": 5, "random_seed": 42})
    mlflow.set_tags({"git_commit": commit, "dvc_raw_md5": data_md5})
    mlflow.log_input(mlflow.data.from_pandas(train, targets="target"))

    model.fit(x_train, y_train)

    info = mlflow.sklearn.log_model(model, name="model",
                                    input_example=x_train.head(3))
    mlflow.log_metric("accuracy", acc, model_id=info.model_id)
```

- `log_params` — konfigūracija, **įskaitant seed'ą**;
- `set_tags` — jungties raktai į Git ir DVC;
- `log_input` — duomenų vardas ir santrauka (digest);
- `log_model` — modelis **ir** aplinkos failai automatiškai.

Kas naujo MLflow 3 — ką pamatysite UI

- **LoggedModel.** Modelis — atskiras objektas su savo `model_id` (`m-…`), ne run'o artefaktas. Run'o artefaktų sąrašas gali būti **tuščias** — modelio failai skiltyje *Models*.
- **Aplinka — nemokamai.** `log_model` įrašo `requirements.txt`, `python_env.yaml`, signatūrą.
- **Registry: aliasai, ne stadijos.** `@champion` / `@challenger` — kilnojamos rodyklės į versiją (stadijos *deprecated*).
- **Nebėra `mlruns/` katalogo.** Su serveriu: metaduomenys — `mlflow.db`, failai — `mlartifacts/`.

Gyvai: trys run'ai

Demo 1 — ką stebėti

1. `dvc repro` — pirmas run'as atsiranda UI su parametrais ir žymomis;
2. dar du paleidimai su kitais parametrais;
3. run'ų lentelė → **Compare** — kuris parametras svarbus?
4. run'o *Models* skiltis — `requirements.txt` viduje;
5. filtras: `metrics.accuracy > 0.9`.

„At scale“: daugelio vartotojų serveris

4. dalis — 2 savaitė

Nuo vieno žmogaus iki komandos

Vieno žmogaus serveris — viena komanda:

```
mlflow server --backend-store-uri sqlite:///mlflow.db
              --artifacts-destination ./mlartifacts
```

Komandai to nepakanka — trys pakeitimai:

Komponentas	**Vieno žmogaus**	**Komandos**
Metaduomenų DB	SQLite failas	**PostgreSQL**
Artefaktai	lokalus katalogas	**MinIO / S3 / GCS**
Prieiga	jokios	**basic-auth + rolės**

Visa tai — vienas `docker-compose.yaml`; kėlimas į Kubernetes — U1 užduotis (8 sav.). Semestro projekte **kiekviena komanda naudoja savo serverį**.

Saugumas — prieš dedant serverį į tinklą

MLflow serveris be autentifikacijos yra atviras visiems

Bet kas, pasiekiantis portą, gali skaityti ir **trinti** eksperimentus; istorijoje būta ir RCE spragų. Viešai eksponuoti serveriai skenuojami automatiškai.

Taisyklės:

- niekada nekelkite pliko serverio į viešą internetą;
- įjunkite basic-auth (nuo 3.13 — ir admin UI);
- naujose versijose serveris pagal nutylėjimą priima tik localhost (`--allowed-hosts` kitiems vardams);
- išsamiai — 13 savaitė (sauga ir atitiktis).

Gyvai: klasės serveris ir LLM mokymas

Demo 2 — ką stebėti

1. tas pats `train.py`, pakeistas tik `MLFLOW_TRACKING_URI` — run'as atsiranda **bendrame** eksperimente;
2. stulpelis `mlflow.user` — kieno kuris run'as;
3. gyvas LLM fine-tuning run'as: `loss` kreivė **ir** `system/gpu_0_*` — GPU apkrova, atmintis;
4. sistemos metrikos atsako į klausimą „ar mano GPU iš viso dirba?“

Praktikoje prie šio serverio prisijungsite ir jūs — Tutorial 2.4.

DVC: duomenų baitai ir pipeline

5. dalis — 2 savaitė

Kodėl Git nepakanka duomenims?

1. 10 GB rinkinys $\times$ 10 versijų $=$ 100 GB repozitorija — Git istorija nesuspaudžia binarinių failų;
2. GitHub atmeta failus >100 MB; Git LFS kvotos mažos ir mokamos;
3. duomenų „diff'as“ žmogui nieko nesako — svarbi tik **tapatybė**.

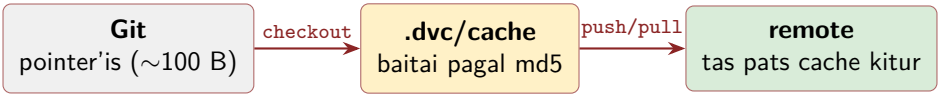
DVC (Data Version Control)

Git plėtinys dideliems failams ir ML pipeline: Git'e — tik **pointer'iai** ($\sim$100 baitų), tikrieji baitai — cache ir **bet kurioje jūsų** saugykloje (katalogas, SSH, MinIO, S3, GCS). Serverio nėra — koordinuoja Git.

Mechanizmas: pointer → cache → remote

`dvc add data/raw/dataset.csv` suskaičiuoja turinio santrauką ir į Git'ą įdeda pointer'į:

```
outs:
- md5: 19a8dabe16b1472ef5516cfdaadb1c6c
  size: 198429
  path: dataset.csv
```



Cache adresuojamas **pagal turinį**: tie patys baitai → ta pati santrauka → saugoma vieną kartą.

Dviejų žmonių scenarijus — du push'ai

Jūs:

```
dvc add data/raw/dataset.csv
git commit
dvc push   # baitai -> remote
git push   # pointer'iai -> Git
```

Kolega:

```
git clone <repo>
dvc pull   # pagal santrauka
dvc repro  # treniruoja
```

`dvc pull` prašo ne „duok failą *vardu*“, o „duok objektą, kurio **santrauka** `19a8dabe…`“ — parsiųsti baitai per-suskaičiuojami.

Pagrindinė mintis

Kolega jūsų duomenimis ne *pasitiki* — jie **sutikrinami pagal santrauką**. Tikslumo garantija yra **matematinė, o ne organizacinė**.

Pipeline: `dvc.yaml` ir pokyčių aptikimas

```yaml
stages:
  prepare:
    cmd: python src/prepare_data.py
    deps: [data/raw/dataset.csv]
    outs: [data/processed/]
  train:
    cmd: python src/train.py
    params:
      - params.yaml:
    outs: [models/model.pkl]
  evaluate:
    metrics:
      - metrics.json:
          cache: false
```

`dvc repro` perskaičiuoja **tik tai, kas pasikeitė**:

Pakeitėte...	**Persileidžia**
nieko	niekas
`max_depth`	train + evaluate
duomenis	visos stadijos
`evaluate.py`	tik evaluate

`dvc.lock` (commit'inamas!) fiksuoja visų įvesčių md5 ir parametrus — tai **lineage įrašas**.

Gyvai: pipeline ir pokyčių aptikimas

Demo 3 — ką stebėti

1. `dvc repro` — visos trys stadijos, MLflow run'as sukuriamas **iš pipeline**;
2. pakartotinis `dvc repro` — „didn't change, skipping“;
3. keičiam vieną parametrą — persileidžia tik `train` + `evaluate`;
4. `dvc metrics diff` — metrikų pokytis tarp commit'ų.

DVC paskirtis — trys klausimai, į kuriuos Git neatsako

Tapatybė
kurie tiksliai baitai
apmokė šitą modelį?
`santrauka dvc.lock`

Platinimas
ar kiti gali
juos gauti?
`dvc pull` + patikra

Atstatymas
ar galiu grįžti
į seną versiją?
`git checkout + dvc checkout`

DVC sąmoningai **nedaro** kito: neseka metrikų UI (tam MLflow), neorkestruoja tarp mašinų (tam 9 savaitė).

W&B, Hydra ir įrankių žemėlapis

6. dalis — 2 savaitė

Weights & Biases — tos pačios sąvokos kitoje kalboje

Debesų (*hosted-first*) platforma (2017; nuo 2025 — CoreWeave); nemokamas akademinis planas. Stiprybės: realaus laiko grafikai, sweep'ai, gradientų histogramos, ataskaitos.

Sąvoka	**MLflow**	**W&B**
Eksperimentų aplankas	experiment	project
Parametrai	`log_params(dict)`	`wandb.init(config=dict)`
Metrika su žingsniu	`log_metric(k, v, step=i)`	`wandb.log({k: v}, step=i)`
Sistemos metrikos	įjungiamos	automatiškai
Kur veikia	jūsų serveris	W&B debesis

Kurso stuburas — MLflow (Apache-2.0, pilnas self-hosting, 6/7/9/11 savaitės skaito jo objektus); **koncepcijas mokėkite abiejose** — praktikos išplėstinėje dalyje tą patį LLM mokymą stebėsite W&B.

Hydra — konfigūracijos dimensija

Run'o konfigūracija **surenkama paleidimo metu**: bazinis YAML + CLI vėliavėlės. Hydra ją daro komponuojamą ir **užrašo į diską**:

```
python train.py model=gemma_lora optim.lr=3e-4     # perrasyti bet ka
python train.py --multirun max_depth=2,5,10        # eksperimentu tinklelis
```

- `conf/` medis: model/ data/ optim/ — kiekviena ašis atskirai;
- kiekvienas run'as įrašo **sukomponuotą** konfigūraciją (`.hydra/config.yaml`) — artefaktas **pagal konstrukciją**;
- sujungimas su tracker'iu — kelios eilutės: `mlflow.log_params(flatten(cfg))`.

Darbo pasidalijimas

Hydra nusprendžia, kokia run'o konfigūracija **yra** (prieš startą); MLflow/W&B fiksuoja, kas **įvyko** (jam bėgant).

Nuosavybės žemėlapis — penkios dimensijos, penki šeimininkai

Git
kodas

DVC
duomenų baitai

Hydra
konfigūracija

MLflow/W&B
įrašas

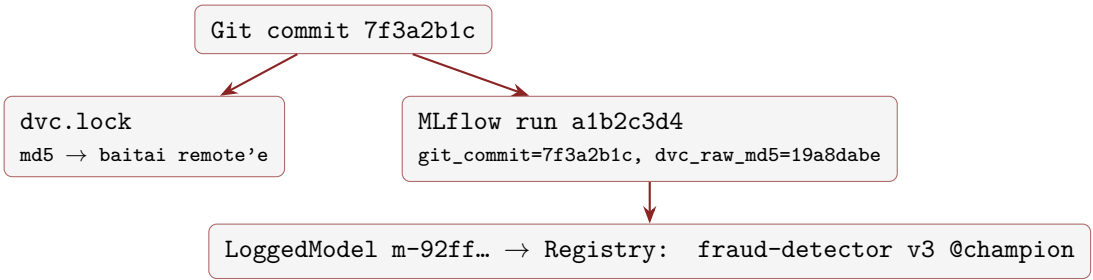
`log_model`
aplinka

	MLflow	**DVC**	**W&B**	**Hydra**
Eksperimentų sekimas	✓✓	✓	✓✓	—
Duomenų **baitai**	tik digest	✓✓	✓	—
Pipeline DAG	—	✓✓	—	—
Konfigūracijų kompozicija	—	params.yaml	—	✓✓
Semestro projekte	**privaloma**	**privaloma**	papildoma	papildoma

Lineage, SLA ir checklist

7. dalis — 2 savaitė

Lineage grandinė — jungties raktai matomi



Git commit — jungties raktas tarp DVC (baitai, receptas) ir MLflow (įrašas, modelio tapatybė). Įrankiai vienas kito nemato natūraliai — jungtį rašote **jūs** (žymos!), o NFR-10 tikrina būtent ją.

Reprodukcijos SLA — atkartojamumas tampa išmatuojamas

SLA

Bet kurį istorinį eksperimentą galima atkurti vienu `python reproduce.py <run_id>` per $<$**2 min.**, o rezultatas — statistiškai ekvivalentus (Δaccuracy $\leq 0{,}005$).

Penki žingsniai:

```
run = mlflow.get_run(run_id)        # 0. irasas: zymos, parametrai
git checkout <run.git_commit>       # 1. to momento kodas + pointer'iai
dvc checkout                        # 2. tie patys duomenu baitai
# 3. params.yaml perrasomas run'o parametrais
dvc repro --force                   # 4. naujas run (zyma reproduces=<id>)
# 5. metriku palyginimas su tolerancija
```

Tai — NFR-08; NFR-09 (atsekamumas) ir NFR-10 (duomenų versija) — praktikoje.

Atkartojamumo checklist — septyni punktai

1. Unikalus `run_name`; **seed — parametras**, ne žyma.
2. Visi hiperparametrai log'inti.
3. Metrikos su `step=` (istorijai) ir galutinės.
4. Modelis per `log_model(name=…, input_example=…)` — aplinka ir signatūra automatiškai.
5. Duomenų kilmė: `log_input` + žyma `dvc_raw_md5`.
6. Git commit žyma.
7. GPU mokymui — įjungtos sistemos metrikos.

Testas

Jei rytoj dingtų jūsų nešiojamas — ar kolega iš repo ir serverio atkurtų jūsų geriausią modelį? Jei ne — kažkurio punkto trūksta.

Ką išsinešti iš šios paskaitos

1. Atkartojamumui reikia versijuoti **penkias dimensijas** — kiekviena turi savo šeimininką.
2. Run'as — žurnalo įrašas; žymos — **jungties raktai** į Git ir DVC.
3. „At scale" = vienas aplinkos kintamasis; saugumas — ne pasirinkimas.
4. DVC garantija **matematinė**, ne organizacinė (turinio adresavimas).
5. MLflow 3: LoggedModel, aliasai, aplinka automatiškai.
6. Atkartojamumas be SLA — tik pažadas; `reproduce.py` jį padaro **išmatuojamą**.

Laboratoriniai

8. dalis — 2 savaitė

Laboratorinis darbas — pagrindinė dalis ($\approx$ 1 val. 45 min.)

	Žingsnis	**Rezultatas**
0	Aplinka	venv su fiksuotomis versijomis; `bootstrap.py`
1	MLflow serveris	pirmas run'as UI; LoggedModel failai
2	DVC	pointer'is Git'e, baitai remote'e; **du push'ai**
3	`dvc.yaml`	pipeline su teisinga sintakse
4	`train.py` + `evaluate.py`	run'as su visomis jungtimis; `dvc.lock`
5	Eksperimentai	$\geq$3 run'ai; `reproduce.py` $<$2 min.
6	Reikalavimai	`requirements.md` v1.1: NFR-08…10

Instrukcijos žingsnis po žingsnio — pratybų medžiagoje (kurso repo). Išplėstinis kelias (5 val.): Hydra, LLM fine-tuning su W&B, nuotoliniai remote'ai.

Deliverables — ką turėsite pabaigoje

- **D1:** veikiantis pipeline — pakartotinis `dvc repro` nieko neperskaičiuoja;
- **D2:** $\geq$3 run'ai su **realiais** hiperparametrais ir jungties žymomis;
- **D3:** `python reproduce.py <run_id>` — $<$2 min., $\Delta \leq 0{,}005$;
- **D4:** `requirements.md` v1.1 + Git tag `v1.1`;
- **D5:** DVC remote sinchronizuotas (bet kuris backend'as).

Savęs patikra

Laboratorinis atskirai nevertinamas — kontrolinis sąrašas ir klausimai pratybų medžiagoje. Visa tai — U1 ir U2 užduočių pamatas.

Ačiū už dėmesį!

Klausimai?